Recruitment Assessment Report

Notice: *This report is generated strictly as decision support for human review. It does not constitute an automatic hiring or rejection decision.*

Candidate Details

* **Name:** AKSHATA KAMBLE

* **Email:** akshatakamble102004@gmail.com

* **Phone:** 7385592127

* **Education:**

* **B.Tech - Computer Science And Business System:** JSPM Rajarshi Shahu College of Engineering, Tathawade, Pune (CGPA: 8.05 | 2024–Present)

* **Diploma - Computer Engineering:** Marathwada Mitra Mandal's Polytechnic, Pune (Percentage: 81.26% | 2020–2023)

* **Class X (SSC Board):** B.R. Gholap Vidyalaya Sangavi, Pune (Percentage: 86.00%)

* **Work Experience:**

* **Web Developer at MidBrain Technologies (Jul 2022 – Aug 2022):** Designed and developed 5+ responsive web pages using HTML, CSS, and JavaScript; built interactive UI components (forms, navigation menus, sliders); performed functional testing; optimized website performance and load speed.

* **Projects:**

* **StudyMate — Personalized AI Learning & Quiz Platform:** Java-based AI app extracting PDF content, generating quizzes using LLM prompts, and offering a study chatbot via OpenRouter API (Technologies: Advanced Java, Swing, MySQL, JDBC, OpenRouter API).

* **ResumeMatch – AI-Powered Resume Job Matching System:** Integrated OpenAI API (GPT-4o-mini) with prompt-based feedback, rule-based fallback, and a Flask app predicting match scores using TF-IDF and Random Forest Regression ($R^2 = 0.70$) (Technologies: Python, Flask, Scikit-learn, SBERT, OpenAI API).

* **AI-Based Movie Review Sentiment Analysis:** Real-time sentiment prediction using NLP preprocessing (tokenization, stopword removal, lemmatization) and TF-IDF vectorization on 50,000 IMDb reviews; achieved 90% accuracy with tuned Naive Bayes (Technologies: Python, Flask, Scikit-learn, Naive Bayes).

* **Certifications:**

* Prompt Engineering – Simplilearn

* Generative AI – IBM

* Artificial Intelligence Fundamentals – IBM

* Machine Learning – IBM

* Python – Great Learning

Skills Assessment

Skills Breakdown

* **Required Skills (Job Description):** Python, Java, R, Scala, PyTorch, TensorFlow, Keras, Scikit-learn, Hugging Face, SpaCy, NLTK, Spark, Hadoop, AWS, Google Cloud Platform, Microsoft Azure.

* **Candidate Skills:** ML Algorithms, Scikit-learn, Pandas, NumPy, Data Preprocessing, Model Evaluation, Deep Learning, Data Structures & Algorithms, OOPs, Exception Handling, File Handling, SQL, MySQL, CRUD Operations, REST APIs, Flask, FastAPI, Large Language Models, Prompt Engineering, Docker, Git, HTML, CSS, JavaScript, Python, Java.

* **Exact Matched Skills (3):** Python, Java, Scikit-learn

* **Related Matched Skills (0):** None

* **Missing Skills (13):** R, Scala, PyTorch, TensorFlow, Keras, Hugging Face, SpaCy, NLTK, Spark, Hadoop, AWS, Google Cloud Platform, Microsoft Azure

Calculated Skills Score

* **Score:** **18.8%**

* **Formula:** `(3 exact x 1 + 0 related x 0.5) / 16 required x 100`

Screening Observations

Strengths

1. **Required Programming Languages:** Demonstrated background in required languages Python and Java (Evidence: Listed under Technical Skills on resume).

2. **Applied ML Framework Experience:** Hands-on machine learning experience using Scikit-learn (Evidence: Listed in Technical Skills and applied in *ResumeMatch* and *AI-Based Movie Review Sentiment Analysis* projects).

3. **Core Computer Science Fundamentals:** Grounding in key computer science foundational topics (Evidence: Resume explicitly cites Data Structures & Algorithms, OOPs, Exception Handling, and File Handling).

Relevant Gaps

1. **Deep Learning Frameworks:** Lack of explicit evidence using required frameworks PyTorch, TensorFlow, or Keras.

2. **Big Data & Cloud Platforms:** No listed experience with required tools Spark, Hadoop, AWS, GCP, or Azure.

3. **NLP Libraries:** Absence of specific NLP libraries requested in the job description, such as Hugging Face, SpaCy, or NLTK.

Verification Areas

1. **Practical Deep Learning Depth:** The resume lists "Deep Learning" as a skill, but primary projects highlight Scikit-learn models and LLM API integrations rather than custom deep learning framework implementation.

2. **Availability & Status:** Academic timeline shows B.Tech status as "2024–Present"; candidate status regarding availability for full-time employment requires verification.

Summary

The candidate possesses fundamental proficiency in Python and Java, along with practical project experience utilizing Scikit-learn, basic NLP techniques, and API-based LLM integrations. However, notable skill gaps exist relative to the job description regarding specific Deep Learning frameworks (PyTorch, TensorFlow, Keras), specialized NLP tools (Hugging Face, SpaCy, NLTK), and Cloud/Big Data technologies (AWS, GCP, Azure, Spark, Hadoop).

Interview Questions

Question 1 (Technical)

* **Question:** In Python's Scikit-learn library, how does `fit_transform()` differ from `transform()` when performing feature scaling or text vectorization?

* **Difficulty:** Beginner

* **Evidence / Skill:** Scikit-learn / Python

* **Expected Answer Points:**

* `fit_transform()` learns statistical parameters (e.g., vocabulary or mean/std) and transforms the training data.

* `transform()` applies previously learned parameters to transform new or test data.

* Using `fit_transform()` on test data causes data leakage from the test set into the model.

Question 2 (Technical)

* **Question:** How does TF-IDF vectorization calculate word importance across a corpus of documents, and how does it address high-frequency common words?

* **Difficulty:** Intermediate

* **Evidence / Skill:** NLP / Scikit-learn / TF-IDF

* **Expected Answer Points:**

* Term Frequency (TF) measures how frequently a term appears in a given document.

* Inverse Document Frequency (IDF) measures how rare or common a term is across the entire corpus.

* Common words across all documents receive a low IDF score, reducing their overall TF-IDF weight.

Question 3 (Technical)

* **Question:** How do Sentence-BERT (SBERT) embeddings differ architectural-wise from standard BERT cross-encoder outputs when performing semantic similarity searches?

* **Difficulty:** Advanced

* **Evidence / Skill:** Sentence-BERT / Cosine Similarity

* **Expected Answer Points:**

* Standard BERT cross-encoders require pair-wise input, making scaling to large datasets computationally expensive $O(N^2)$.

* SBERT uses a Siamese or triplet network architecture to derive fixed-size, semantically meaningful sentence embeddings.

* SBERT embeddings allow fast similarity computation via cosine similarity or vector indexes.

Question 4 (Project)

* **Question:** In your AI-Based Movie Review Sentiment Analysis project, what text preprocessing steps did you perform before feeding features into the Naive Bayes model?

* **Difficulty:** Beginner

* **Evidence / Skill:** AI-Based Movie Review Sentiment Analysis Project

* **Expected Answer Points:**

* Preprocessed raw IMDb text using tokenization, stopword removal, and lemmatization.

* Converted text into numerical feature vectors using TF-IDF vectorization.

* Evaluated model performance using metrics like accuracy and F1-score (achieving 90% accuracy).

Question 5 (Project)

* **Question:** In the ResumeMatch project, how did you implement the automatic rule-based fallback mechanism to handle OpenAI API rate limits or connection errors?

* **Difficulty:** Intermediate

* **Evidence / Skill:** ResumeMatch Project

* **Expected Answer Points:**

* Handled API exceptions, timeouts, or rate limit errors explicitly during network calls.

* Fell back to a deterministic rule-based ATS score or offline model (TF-IDF + Random Forest / SBERT).

* Maintained application availability and provided valid score outputs despite upstream API disruptions.

Question 6 (Project)

* **Question:** In your StudyMate project, how did you manage PDF text extraction, chunking, and context passing to the OpenRouter API without hitting token context limits?

* **Difficulty:** Advanced

* **Evidence / Skill:** StudyMate Project

* **Expected Answer Points:**

* Extracted text content from uploaded PDFs using Java libraries.

* Partitioned continuous document text into discrete chunks suitable for LLM prompt context.

* Constructed prompts including context blocks to generate context-aware study chatbot answers via OpenRouter API.